4.2 Refactoring Using AI Tools

Now that we've briefly dealt with the theory of refactoring, it's time to work with the source code. First of all, you'll learn how to assess whether you should revise a piece of source code at all, the criteria for making this decision, and how AI tools can help you. You'll also learn some strategies and best practices for specific refactoring processes.

4.2.1 Recognizing the Need for Refactoring

Your time as a developer is very valuable, so you shouldn't waste it on unnecessary work. It's therefore important to first find out whether refactoring is expedient and where the weaknesses in the code can be found.

There are numerous tools available for this type of analysis, such as SonarQube, which perform a static code analysis. These tools can provide a good overview of the overall system. This is also the weakness of many AI tools, as they rarely have an overview of the entire context of your application, but only smaller parts. A combination of different tools is therefore ideal: classic static code analysis for the overview and AI tools for the details and the actual refactoring. You can also use the AI tools as a kind of code reviewer that uncovers refactoring potential.

Code Smells

You can take the term *code smells* almost literally. Code that has code smells harbors potential problems that you can fix with refactoring. Code smells aren't necessarily bugs, but if you ignore them for too long, they can cause serious problems. Following are typical code smells:

- **Duplicate code**
  The code has the same or similar blocks in different places. If you correct an error at one point, you must also do this at all other points.

Maintainability is therefore reduced, and the potential for errors is high.

- **Long methods**
  Functions and methods should be as compact as possible and only deal with one aspect. The more lines a function has, the greater the potential complexity.
- **Large classes**
  The same applies to classes as to functions and methods. If a class is too large, measured by the number of lines, properties, or methods, there is a high probability that it will do too much.
- **Feature envy**
  This type of code smell refers to a method that accesses more properties of another class than its own. This is an indication that the method is wrong at this point.
- **Data clumps**
  These are groups of variables that are used in the same way at different places in an application. They should be combined in a separate class or data structure.

Code smells are present in various forms in almost every application, but if you use the right tools, you can easily detect and rectify them. For example, we use a TypeScript class named `OrderProcessor`, which reads as follows:

```
class OrderProcessor {
  orders: any[] = [];

  addOrder(order: any) {
    this.orders.push(order);
    console.log("Order added:", order);
    if (this.orders.length > 100) {
      console.log("Too many orders");
    }
  }

  processOrders() {
    for (let i = 0; i < this.orders.length; i++) {
      let order = this.orders[i];
      console.log("Processing order:", order);
```

```
        if (order.status === "NEW") {
          console.log("Order is new");
        } else if (order.status === "PROCESSING") {
          console.log("Order is being processed");
        } else if (order.status === "COMPLETED") {
          console.log("Order is completed");
        } else {
          console.log("Unknown order status");
        }

        if (order.customer.address.city === "Berlin") {
          console.log("Customer is from Berlin");
        }
      }
    }

    generateReport() {
      let report = "Order Report\n";
      report += "=============\n";

      for (let i = 0; i < this.orders.length; i++) {
        let order = this.orders[i];
        report += `Order ${i + 1}: ${order.id}\n`;

        if (order.customer.address.city === "Berlin") {
          report += "Customer is from Berlin\n";
        }
      }

      if (this.orders.length > 100) {
        report += "Warning: Too many orders\n";
      }

      console.log(report);
    }
  }
```

The class has the `orders` property in which it saves orders. The `addOrder` method adds new orders, the `processOrder` method processes the orders, and the `generateReport` method generates a report. You can use almost any AI tool for analysis. For example, you could copy the code into the Gemini console and ask for an analysis.

In this example, we use GitHub Copilot and Visual Studio Code (VS Code) integration. Here you can use the combination of the chat feature and workspace to analyze the code. To do this, open the class in the development environment, and enter the following prompt in the chat window:

Prompt: @workspace /explain Does the `OrderProcessor` class contain code smells?

> Yes, the `OrderProcessor` class contains several code smells. Let's go through them step by step:
>
> [explanation of code smells in the code]
>
> Here is a refactored version of the `OrderProcessor` class addressing these issues:
>
> ```
> interface Order {
>   id: number;
>   status: 'NEW' | 'PROCESSING' | 'COMPLETED';
>   customer: {
>     address: {
>       city: string;
>     };
>   };
> }
>
> class OrderProcessor {
>   ... updated code ...
> }
> ```
>
> **Changes made:**
>
> [explanation of changes made in the code]
>
> These changes improve the readability, maintainability, and robustness of the code.

This prompt is kept very general so that you give the tool plenty of scope to analyze the code and find code smells. Regardless of which tool you use, it's

important with such a prompt that you make the code available as comprehensively as possible and not just individual sections. The more context you give the AI tool, the better it can do its job.

In the prompt, you explicitly asked for code smells. This is an established term that is sufficiently described in the literature so that AI can work with it. The use of such technical terms, such as certain design patterns, can make the prompts much more compact, as you don't have to describe the problem in a complex way. Try to use technical terms in your prompts so that you describe what you want to achieve as precisely as possible.

GitHub Copilot has found some vulnerabilities in our code. The tool identifies the code smells, explains what the problem is, and also provides an improved version of the class, including a detailed explanation of the changes. The following code smells were found and fixed:

- **Use of the `any` type**
  If you use the `any` type in TypeScript, you lose the advantages of the static type system. GitHub Copilot has created the `order` interface for the order and uses it throughout the methods and properties of the class.
- **Use of magic numbers**
  You should avoid storing numbers with a specific meaning, such as the limit for orders, directly in the code. Copilot provides a better solution here by storing the number in the form of the `MAX_ORDERS` property.
- **Repetitions in the code**
  The check as to whether the city in the order is Berlin can be found twice in the code and can be outsourced.
- **Long methods**
  The `processOrders` and `generateReport` methods are relatively long. Copilot has simplified the iteration of the orders by using the `forEach` method and has also outsourced parts of the methods so they are more compact and easier to read.
- **Missing error handling**
  When accessing properties such as `city` in the order city check, the code assumes that all properties in the chain exist. If that isn't the case, the JavaScript engine will throw an exception. For this reason, you should install an error handling routine here.

- **Console outputs**
  Logging using the `console.log` method isn't always the best option. Ideally, you should use your own logger for this, whose behavior you can better control.

The information provided by GitHub Copilot is very good, and the generated result is also significantly better than the original version of the source code. However, some points of criticism aren't taken into account (or only partially so) in the result, so you still have some work to do. The duplicate code was only moved to the `checkCustomerCity` method in one place. There is also no solution for using the logger, although the application-wide context is also missing here.

One of the code smells that Copilot didn't find is that the class covers too many responsibilities. Managing orders and generating a report don't necessarily have anything to do with each other and could be outsourced to separate classes. Access to the properties of the order address is an example of feature envy. The `OrderProcessor` shouldn't access the address object directly. Instead, it's better to introduce a separate class that provides you with the corresponding methods.

You can also be much more specific if, for example, you have a class or function that you're not happy with but don't know exactly how to modify it. The following function contains unnecessary duplicates in the code that the AI tool should resolve:

```
function processOrders(orders) {
  let totalRevenue = 0;
  let totalDiscountedRevenue = 0;
  let highValueOrders = [];
  let discountedOrders = [];

  for (let i = 0; i < orders.length; i++) {
    const order = orders[i];

    if (order.status === "completed") {
      totalRevenue += order.price;
    }
    if (order.status === "completed" && order.discount
```

```
> 0) {
      totalDiscountedRevenue += order.price -
order.discount;
      discountedOrders.push(order);
    }
    if (order.status === "completed" && order.price >
100) {
      if (order.discount > 0) {
        discountedOrders.push(order);
      }
      highValueOrders.push(order);
    }
  }

  return {
    totalRevenue,
    totalDiscountedRevenue,
    highValueOrders,
    discountedOrders,
  };
}
```

In this code example, you check whether the status of the order is `completed` in each `if` block. Such code is created quickly if you translate the requirements directly into code and don't make any optimizations. However, ChatGPT or another tool can do this for us with the following prompt:

Prompt: Could you please reduce the code duplicates in the following function:

[function code]

> You can reduce the code duplication in the `processOrders` function by combining the conditions that check for the order's status and applying the relevant logic inside a single block. Here's a refactored version of your function:
>
> ```
> function processOrders(orders) {
>   let totalRevenue = 0;
>   let totalDiscountedRevenue = 0;
> ```

```
    let highValueOrders = [];
    let discountedOrders = [];

    for (const order of orders) {
      if (order.status !== "completed") continue;

      totalRevenue += order.price;

      if (order.discount > 0) {
        totalDiscountedRevenue += order.price -
  order.discount;
        discountedOrders.push(order);
      }

      if (order.price > 100) {
        highValueOrders.push(order);
      }
    }
    return {
      totalRevenue,
      totalDiscountedRevenue,
      highValueOrders,
      discountedOrders,
    };
  }
```

Changes made: [explanation of the changes made]

The AI tool has rewritten the code so that the multiple checks of the order status have been summarized. If the status doesn't correspond to the `"completed"` value, the system continues directly with the next run. This makes the code more compact and easier to read.

So, if you're not satisfied with your source code and know what changes you want to see, you can formulate this in a prompt, and the AI tool will adapt your code accordingly. Of course, there's also code that can't be optimized so easily. In this case, you should check the adjustments very carefully, as AI tools tend to make changes that don't really improve the code.

Anti-Patterns

You've probably heard of *design patterns*. These are established and reusable solutions for frequently occurring problems in software development. *Anti-patterns* are, to a certain extent, the opposite. They're often patterns or practices in the source code that cause problems in the long term. These solutions are usually the result of good intentions, but turn out to be problematic during further development. Typical anti-patterns include the following, for example:

- **God object**
  The god object or the god class is, in a way, the further development of a large class. It's a central component of an application. Over the course of development, more and more aspects have been added to the class, so that it takes care of too many aspects. This makes the source code much more difficult to maintain.
- **Incorrect use of the singleton pattern**
  In general, the singleton pattern has a bad reputation because it can easily be used incorrectly. For example, it can be used to manage the global status of an application.
- **Spaghetti code**
  If source code has a poor or inadequate structure, you can compare it to a mountain of spaghetti: you pull on one end and the whole mountain moves. This kind of source code makes it very difficult to correct errors or develop features any further.
- **Golden hammer**
  This anti-pattern refers to the fact that an established solution is used for all possible problems, even if it's not suitable for them.
- **Premature optimization**
  In applications, you often encounter the problem that the code is overengineered; that is, the solutions are unnecessarily complicated and solve problems that haven't even arisen yet.

The `UserService` TypeScript class serves as the basis for the analysis and subsequent improvement:

```
class UserService {
  private static instance: UserService;
```

```
  private constructor() {}

  static getInstance(): UserService {
    if (!UserService.instance) {
      UserService.instance = new UserService();
    }
    return UserService.instance;
  }

  processUserData(users: any[]): void {
    users.forEach((user) => {
      if (user.isActive) {
        console.log(`Processing active user:
${user.name}`);
        if (user.role === "admin") {
          console.log("Admin privileges granted.");
          if (
            user.lastLogin >
            new Date().getTime() - 1000 * 60 * 60 * 24
          ) {
            console.log("Recent login detected.");
          } else {
            console.log("Login outdated.");
          }
        } else if (user.role === "user") {
          console.log("Standard user.");
          user.permissions.forEach((perm: any) => {
            if (perm.isEnabled) {
              console.log(`Enabled permission:
${perm.name}`);
            }
          });
        } else {
          console.log("Unknown role.");
        }
      } else {
        console.log(`Inactive user: ${user.name}`);
      }
    });
  }

  notifyUsers(users: any[]): void {
```

```
    this.processUserData(users);
  }

  calculateDiscounts(users: any[]): void {
    const discountCache: { [key: string]: number } =
{};

    users.forEach((user) => {
      if (discountCache[user.id]) {
        console.log(
          `Cached discount for user ${user.name}: ${
            discountCache[user.id]
          }`
        );
      } else {
        const discount = user.purchaseHistory.length *
0.05;
        discountCache[user.id] = discount;
        console.log(
          `Calculated discount for user ${user.name}:
             ${discount}`
        );
      }
    });
  }

  logUserActivity(user: any): void {
    console.log(`Logging activity for user:
${user.name}`);
  }

  generateUserReport(users: any[], reportType:
string): void {
    console.log(
      `Generating ${reportType} report for
       ${users.length} users.`
    );
  }
}
```

As in the preceding example, we use GitHub Copilot in VS Code for refactoring with regard to anti-patterns. You can use the Copilot chat integration

to submit the following prompt to analyze the code of the class:

Prompt: @workspace /explain Does the code of the `UserService` class contain anti-patterns?

> Yes, the `UserService` class contains several anti-patterns and areas for improvement. Let's go through them step by step.
>
> [explanation of anti-patterns in the code]
>
> **Refactored example:**
>
> Here's a refactored version addressing some of these issues:

```
interface User {
  id: string;
  name: string;
  isActive: boolean;
  role: 'admin' | 'user' | 'unknown';
  lastLogin: number;
  permissions: Permission[];
  purchaseHistory: any[];
}

interface Permission {
  name: string;
  isEnabled: boolean;
}

class UserService {
  private static instance: UserService;

  private constructor() {}

  static getInstance(): UserService {
    if (!UserService.instance) {
      UserService.instance = new UserService();
    }
    return UserService.instance;
  }
```

```
  processUserData(users: User[]): void {
    users.forEach((user) => {
      if (user.isActive) {
        this.processActiveUser(user);
      } else {
        console.log(`Inactive user: ${user.name}`);
      }
    });
  }

  private processActiveUser(user: User): void {
    console.log(`Processing active user:
${user.name}`);
    switch (user.role) {
      case 'admin':
        this.processAdminUser(user);
        break;
      case 'user':
        this.processStandardUser(user);
        break;
      default:
        console.log("Unknown role.");
    }
  }

  private processAdminUser(user: User): void {
    console.log("Admin privileges granted.");
    const oneDayInMillis = 1000 * 60 * 60 * 24;
    if (user.lastLogin > new Date().getTime() -
oneDayInMillis) {
      console.log("Recent login detected.");
    } else {
      console.log("Login outdated.");
    }
  }

  private processStandardUser(user: User): void {
    console.log("Standard user.");
    user.permissions.forEach((perm) => {
      if (perm.isEnabled) {
        console.log(`Enabled permission:
${perm.name}`);
      }
```

```
    });
  }

  notifyUsers(users: User[]): void {
    this.processUserData(users);
  }
  calculateDiscounts(users: User[]): void {
    const discountCache: { [key: string]: number } =
{};

    users.forEach((user) => {
      if (discountCache[user.id]) {
        console.log(`Cached discount for user
${user.name}:
${discountCache[user.id]}`);
      } else {
        const discount = user.purchaseHistory.length *
0.05;
        discountCache[user.id] = discount;
        console.log(`Calculated discount for user
${user.name}:
${discount}`);
      }
    });
  }

  logUserActivity(user: User): void {
    console.log(`Logging activity for user:
${user.name}`);
  }

  generateUserReport(users: User[], reportType:
string): void {
    console.log(`Generating ${reportType} report for
${users.length} users.`);
  }
}
```

Summary: . . . By addressing these points, the code becomes more maintainable, readable, and testable.

Once again, GitHub Copilot has done a great job, finding some weaknesses in the code and suggesting an alternative implementation:

- **Singleton pattern**
  Copilot has recognized the singleton pattern in the code and points out that you should be careful when using it.
- **Type safety**
  The original code used the `any` type. Copilot has created a `User` interface and uses it throughout the class.
- **Single responsibility principle**
  The `processUserData` method performs too many tasks. In its alternative implementation, Copilot has neatly outsourced the responsibilities to its own methods.
- **Reduce nesting**
  When nesting, the `processUserData` method is again noticeable in a negative way. By splitting the method, Copilot has also solved this problem. The combination of the violation of the single responsibility principle and the poor structuring indicates that we're dealing with spaghetti code.
- **Magic numbers**
  The calculation of the number of milliseconds in a day isn't obvious at first glance. In the revised version of the code, this calculation is stored in an appropriately named variable.
- **Logging**
  In its analysis, Copilot notices that `console.log` is used in the class and suggests using a separate logger instead.
- **Error handling**
  Another point of criticism in the analysis is the lack of error handling.

As with the analysis of the code smells, Copilot gives us a mixture of a concrete alternative implementation and some hints for the anti-patterns that we have to take care of ourselves. You can use the result of this prompt in your application without hesitation, as it significantly improves the existing code in any case.

The original code has several other vulnerabilities that haven't been addressed:

- **Too many responsibilities**
  The class covers too many responsibilities at once. The `logUserActivity`, `generateUserReport`, or `calculateDiscounts` methods provide information on this.
- **Premature optimization**
  The cache in the `calculateDiscounts` method could be an indication of an unnecessary optimization that increases the complexity of the code. At the very least, it would be good to have a corresponding note at such a point.
- **Inappropriate use of a method**
  The `notifyUsers` method is merely an alias for `processUserData`. Such code sections can indicate an anti-pattern.

When working with existing code, there are some best practices that can help you avoid problems. You can find out how to proceed in the following sections.